

CUSTOMER SHOPPING BEHAVIOR ANALYSIS

End-to-End Project Report

*A Step-by-Step Walkthrough: Environment Setup • Python & Jupyter • Data Cleaning • PostgreSQL
Analysis • Power BI Dashboard*

PRESENTED BY : MANAHIL SAEED

Dataset: 3,900 customer transactions across 18 features

Tools Used: VS Code · Python · JupyterLab · pandas · PostgreSQL / pgAdmin4 · Power BI

Report compiled July 2026

Index

Index.....	2
1. Project Overview.....	4
1.1 Dataset Summary.....	4
1.2 Workflow at a Glance.....	4
2. Environment Setup	5
2.1 Step 1: Install VS Code and Python	5
2.2 Step 2: Verify the Python Installation	5
2.3 Step 3: Install JupyterLab and Analysis Packages	5
2.4 Step 4: Launch JupyterLab	6
3. Data Loading and Initial Exploration	8
3.1 Loading the Dataset	8
3.2 Initial Structural Checks	8
4. Data Cleaning and Preparation	12
4.1 Handling Missing Data	12
4.2 Column Standardization (Snake Case)	12
4.3 Feature Engineering.....	13
4.4 Data Consistency Check	14
5. Database Integration — Loading into PostgreSQL.....	17
6. Data Analysis Using SQL (Business Questions)	18
6.1 Revenue by Gender.....	18
6.2 High-Spending Discount Users.....	19
6.3 Top 5 Products by Rating	19
6.4 Shipping Type Comparison	20
6.5 Subscribers vs. Non-Subscribers	21
6.6 Discount-Dependent Products.....	22
6.7 Customer Segmentation	22
6.8 Top 3 Products per Category	23
6.9 Repeat Buyers & Subscriptions.....	24
6.10 Revenue by Age Group	25
7. Dashboard Development in Power BI.....	26
7.1 Building the Report Canvas.....	26
7.2 Formatting with Table Tools	26

7.3 Creating Measures and KPI Cards	27
7.4 Final Interactive Dashboard	27
7.5 Dashboard Highlights	28
8. Business Recommendations	29
9. Conclusion	30

1. Project Overview

This report documents the complete, step-by-step process behind the Customer Shopping Behavior Analysis project — from setting up the development environment through to the final interactive Power BI dashboard. It combines the working process (captured through screenshots taken at each stage) with the resulting analysis and business insights, so that every step of the workflow can be followed and reproduced in order.

The project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior in order to guide strategic business decisions.

1.1 Dataset Summary

- Rows: 3,900
- Columns: 18
- Customer demographics: Age, Gender, Location, Subscription Status
- Purchase details: Item Purchased, Category, Purchase Amount, Season, Size, Color
- Shopping behavior: Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type
- Missing Data: 37 values found in the Review Rating column

1.2 Workflow at a Glance

- Step 1 — Set up the development environment (VS Code, Python)
- Step 2 — Install JupyterLab and required Python packages
- Step 3 — Load and explore the raw dataset in Jupyter
- Step 4 — Clean the data and engineer new features
- Step 5 — Load the cleaned dataset into PostgreSQL
- Step 6 — Answer 10 business questions using SQL in pgAdmin4
- Step 7 — Build an interactive dashboard in Power BI
- Step 8 — Summarize findings into business recommendations

2. Environment Setup

Before any analysis could begin, the local development environment had to be prepared. This involved installing a code editor, the Python interpreter, and the JupyterLab environment used for the exploratory data analysis.

2.1 Step 1: Install VS Code and Python

Visual Studio Code was installed as the primary code editor, and Python was installed as the runtime used for all data-processing scripts.

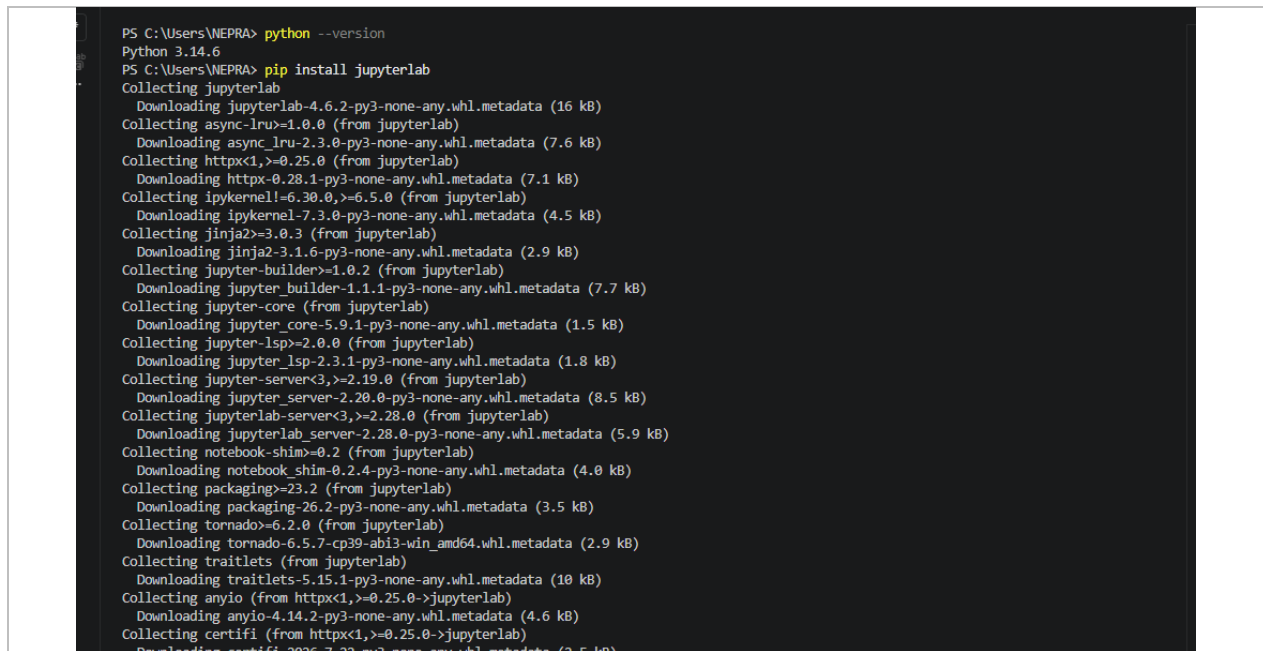
2.2 Step 2: Verify the Python Installation

After installation, the setup was verified directly inside VS Code:

- Closed and reopened VS Code to ensure the installation registered correctly
- Opened the integrated terminal

```
python --version
```

The terminal correctly returned the installed Python version, confirming the interpreter was available on the system PATH.



```
PS C:\Users\NEPRA> python --version
Python 3.14.6
PS C:\Users\NEPRA> pip install jupyterlab
Collecting jupyterlab
  Downloading jupyterlab-4.6.2-py3-none-any.whl.metadata (16 kB)
Collecting async-lru>=1.0.0 (from jupyterlab)
  Downloading async_lru-2.3.0-py3-none-any.whl.metadata (7.6 kB)
Collecting httpx<1,>=0.25.0 (from jupyterlab)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting ipykernel>=6.30.0,<=6.5.0 (from jupyterlab)
  Downloading ipykernel-7.3.0-py3-none-any.whl.metadata (4.5 kB)
Collecting jinja2>=3.0.3 (from jupyterlab)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting jupyter-builder>=1.0.2 (from jupyterlab)
  Downloading jupyter_builder-1.1.1-py3-none-any.whl.metadata (7.7 kB)
Collecting jupyter-core (from jupyterlab)
  Downloading jupyter_core-5.9.1-py3-none-any.whl.metadata (1.5 kB)
Collecting jupyter-lsp>=2.0.0 (from jupyterlab)
  Downloading jupyter_lsp-2.3.1-py3-none-any.whl.metadata (1.8 kB)
Collecting jupyter-server<3,>=2.19.0 (from jupyterlab)
  Downloading jupyter_server-2.20.0-py3-none-any.whl.metadata (8.5 kB)
Collecting jupyterlab_server<3,>=2.28.0 (from jupyterlab)
  Downloading jupyterlab_server-2.28.0-py3-none-any.whl.metadata (5.9 kB)
Collecting notebook-shim>=0.2 (from jupyterlab)
  Downloading notebook_shim-0.2.4-py3-none-any.whl.metadata (4.0 kB)
Collecting packaging>=23.2 (from jupyterlab)
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting tornado>=6.2.0 (from jupyterlab)
  Downloading tornado-6.5.7-cp39-abi3-win_amd64.whl.metadata (2.9 kB)
Collecting traitlets (from jupyterlab)
  Downloading traitlets-5.15.1-py3-none-any.whl.metadata (10 kB)
Collecting anyio (from httpx<1,>=0.25.0->jupyterlab)
  Downloading anyio-4.14.2-py3-none-any.whl.metadata (4.6 kB)
Collecting certifi (from httpx<1,>=0.25.0->jupyterlab)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
```

Figure 2.1 — Verifying the Python installation from the VS Code terminal (`python --version`).

2.3 Step 3: Install JupyterLab and Analysis Packages

With Python confirmed, JupyterLab and the core data-analysis libraries were installed from the same terminal:

```
pip install jupyterlab
```

```
pip install pandas numpy matplotlib sqlalchemy psycopg2-binary openpyxl
```

This installed pandas and numpy for data manipulation, matplotlib for visualization, and sqlalchemy with psycopg2-binary to enable a later connection from Python to a PostgreSQL database.

```
PS C:\Users\NEPRA>
PS C:\Users\NEPRA>
PS C:\Users\NEPRA> pip install pandas numpy matplotlib sqlalchemy psycopg2-binary openpyxl
Collecting pandas
  Downloading pandas-3.0.5-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy
  Downloading numpy-2.5.1-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.1-cp314-cp314-win_amd64.whl.metadata (80 kB)
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.51-cp314-cp314-win_amd64.whl.metadata (9.8 kB)
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2026.3)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
```

Figure 2.2 — Installing JupyterLab via pip in the terminal.

```
Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl (232 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl (2.3 MB)
7.2/7.2 MB 686.4 kB/s 0:00:03
Downloading greenlet-3.5.4-cp314-cp314-win_amd64.whl (248 kB)
Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl (75 kB)
Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl (7.2 MB)
7.2/7.2 MB 681.0 kB/s 0:00:11
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: pyparsing, psycopg2-binary, pillow, numpy, kiwisolver, greenlet, fonttools, et-xmlfile, cycler, sqlalchemy, pandas, openpyxl, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 et-xmlfile-2.0.0 fonttools-4.63.0 greenlet-3.5.4 kiwisolver-1.5.0 matplotlib-3.11.1 numpy-2.5.1 openpyxl-3.1.5 pandas-3.0.5 pillow-12.3.0 psycopg2-binary-2.9.12 pyparsing-3.3.2 sqlalchemy-2.0.51
PS C:\Users\NEPRA> jupyter lab
jupyter : The term 'jupyter' is not recognized as the name of a cmdlet, function, script file, or operable program.
Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ jupyter lab
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (jupyter:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
PS C:\Users\NEPRA>
```

Figure 2.3 — Installing pandas, numpy, matplotlib, sqlalchemy, psycopg2-binary, and openpyxl.

2.4 Step 4: Launch JupyterLab

JupyterLab was then launched from the terminal:

```
jupyter lab
```

This opened a local JupyterLab server in the browser at <http://localhost:8888/lab>, providing the notebook interface used for the exploratory data analysis in the next section.

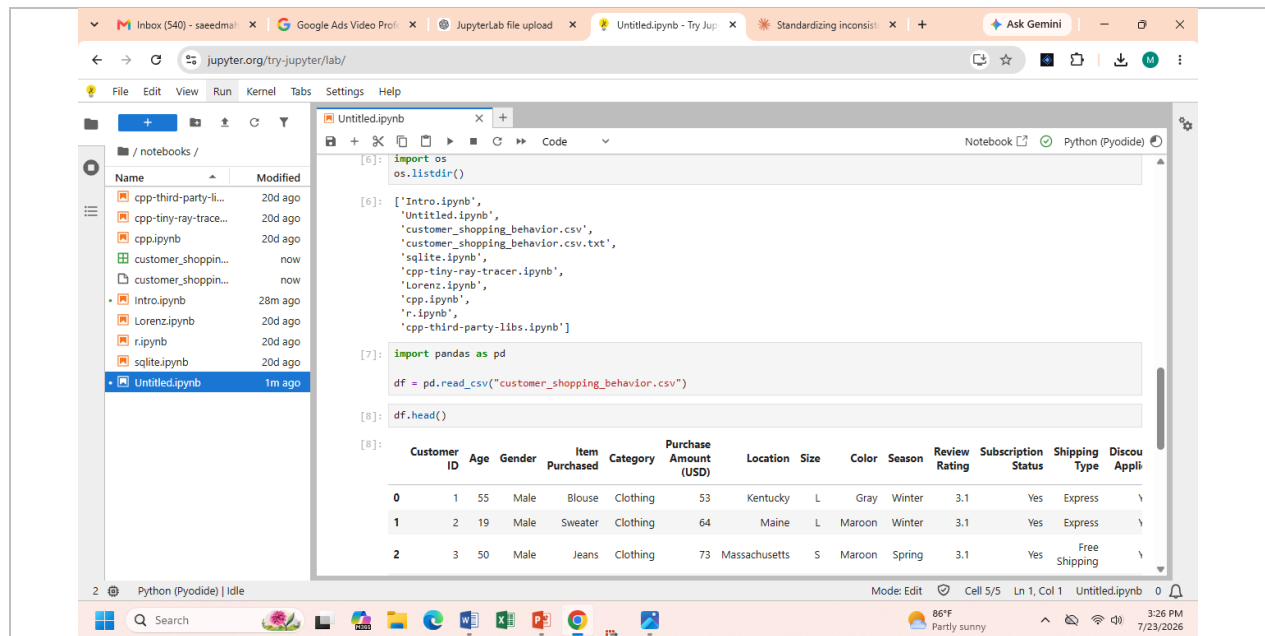


Figure 2.4 — JupyterLab launching and opening in the browser at localhost:8888/lab.

3. Data Loading and Initial Exploration

With the environment ready, the raw customer shopping dataset (3,900 rows × 18 columns) was loaded into a Jupyter notebook using pandas, and an initial exploration was carried out to understand its structure, data types, and summary statistics.

3.1 Loading the Dataset

The dataset was imported using pandas' `read_csv` function into a `DataFrame`, which formed the basis for all subsequent cleaning and analysis steps.

3.2 Initial Structural Checks

`df.info()` was used to inspect column data types and non-null counts, and `.describe()` was used to generate summary statistics (count, mean, standard deviation, min/max, and quartiles) for the numeric fields — including Age, Purchase Amount, Review Rating, and Previous Purchases.

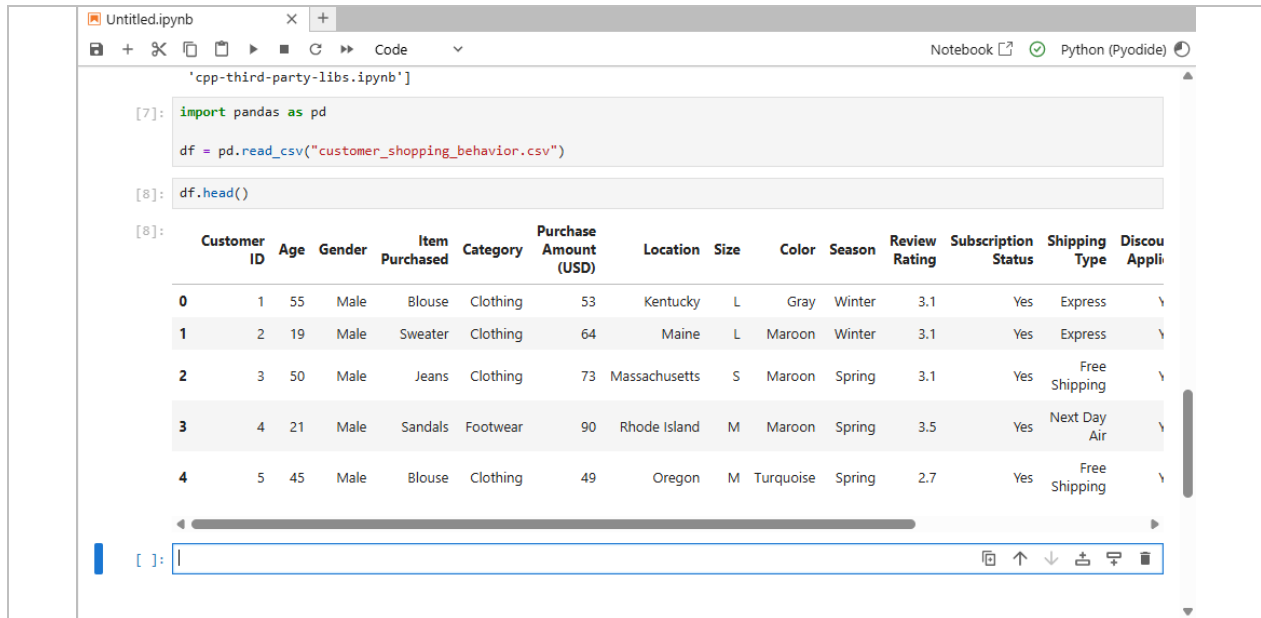


Figure 3.1 — Loading the dataset into a pandas DataFrame in JupyterLab.

```

[9]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID           3900 non-null   int64
1   Age                   3900 non-null   int64
2   Gender                3900 non-null   str
3   Item Purchased        3900 non-null   str
4   Category              3900 non-null   str
5   Purchase Amount (USD) 3900 non-null   int64
6   Location              3900 non-null   str
7   Size                  3900 non-null   str
8   Color                 3900 non-null   str
9   Season                3900 non-null   str
10  Review Rating         3863 non-null   float64
11  Subscription Status   3900 non-null   str
12  Shipping Type         3900 non-null   str
13  Discount Applied      3900 non-null   str
14  Promo Code Used       3900 non-null   str
15  Previous Purchases    3900 non-null   int64
16  Payment Method        3900 non-null   str
17  Frequency of Purchases 3900 non-null   str

```

Figure 3.2 — Running `df.info()` to inspect column types and non-null counts.

```

[10]: df.describe()

#   Customer ID           Age  Purchase Amount (USD)  Review Rating  Previous Purchases
count  3900.000000  3900.000000          3900.000000      3863.000000          3900.000000
mean   1950.500000   44.068462          59.764359       3.750065           25.351538
std    1125.977353   15.207589          23.685392       0.716983           14.447125
min     1.000000    18.000000          20.000000       2.500000           1.000000
25%    975.750000   31.000000          39.000000       3.100000           13.000000
50%    1950.500000  44.000000          60.000000       3.800000           25.000000
75%    2925.250000  57.000000          81.000000       4.400000           38.000000
max    3900.000000  70.000000         100.000000       5.000000           50.000000

[11]: df.describe(include='all')

#   Customer ID           Age  Gender  Item Purchased  Category  Purchase Amount (USD)  Location  Size  Color  Season  Review Rating  Subscription Status  Sh

```

Figure 3.3 — Running `df.describe()` to generate summary statistics for numeric columns.

Figure 3.4 shows the output of the `df.describe(include='all')` command in a Jupyter Notebook. The resulting table provides a comprehensive overview of the dataset's distribution and characteristics.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Standard
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	

Figure 3.4 — Reviewing the resulting summary statistics table.

Figure 3.5 shows the output of the `df.isnull().sum()` command in a Jupyter Notebook. This output provides a detailed view of the dataset's structure, including the data type for each column and the count of missing values.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
dtype	int64	int64	object	object	object	float64	object	object	object	object	float64	object	object	object	object	int64	object	int64
sum	0	0	0	0	0	0	0	0	0	0	37	0	0	0	0	0	0	0

Figure 3.5 — Further inspection of the dataset's structure and sample records.

The screenshot shows a Jupyter Notebook interface with the following content:

```

Payment Method      0
Frequency of Purchases  0
dtype: int64

[17]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

[18]: df.isnull().sum()

[18]: Customer ID      0
Age      0
Gender    0
Item Purchased  0
Category  0
Purchase Amount (USD)  0
Location  0
Size      0
Color     0
Season    0
Review Rating      0
Subscription Status  0
Shipping Type      0
Discount Applied   0
Promo Code Used    0
Previous Purchases  0
Payment Method     0
Frequency of Purchases  0
dtype: int64

```

The notebook interface includes a toolbar at the top with icons for file operations and a status bar at the bottom indicating 'Mode: Edit', 'Cell 11/11', 'Ln 1, Col 1', and the filename 'Untitled.ipynb'.

Figure 3.6 — Continued exploratory checks on the raw dataset.

At this stage the dataset was confirmed to contain 3,900 rows and 18 columns, with 37 missing values isolated to the Review Rating column — setting up the cleaning steps that followed.

4. Data Cleaning and Preparation

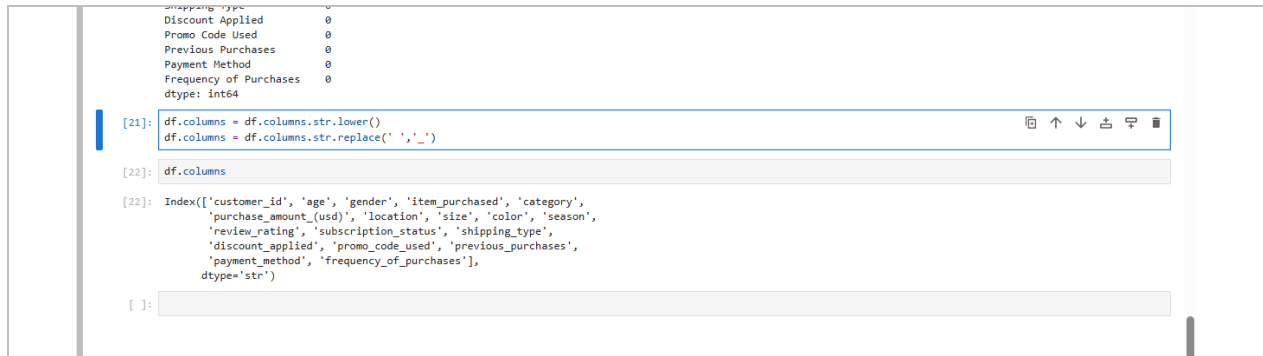
Once the raw structure of the data was understood, a series of cleaning and preparation steps were carried out in pandas to make the dataset consistent and analysis-ready.

4.1 Handling Missing Data

The dataset was checked for null values. The 37 missing entries in the Review Rating column were imputed using the median rating of each product category, preserving the category-level rating distribution rather than applying a single global average.

4.2 Column Standardization (Snake Case)

All column names were renamed to snake_case (for example, converting “Purchase Amount (USD)” to purchase_amount_usd) to improve readability, ensure consistency, and make the columns easier to reference in both Python and SQL.



```

Shipping Type      0
Discount Applied   0
Promo Code Used    0
Previous Purchases 0
Payment Method     0
Frequency of Purchases 0
dtype: int64

[21]: df.columns = df.columns.str.lower()
      df.columns = df.columns.str.replace(' ', '_')

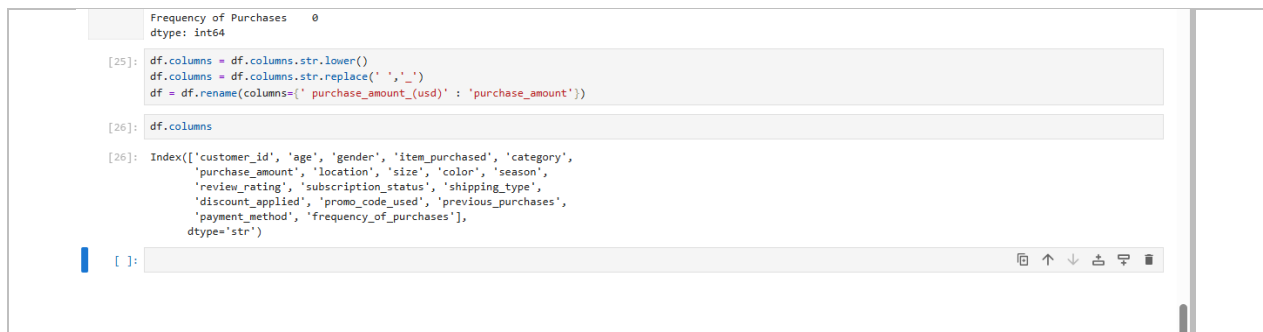
[22]: df.columns

[22]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
          'purchase_amount_usd', 'location', 'size', 'color', 'season',
          'review_rating', 'subscription_status', 'shipping_type',
          'discount_applied', 'promo_code_used', 'previous_purchases',
          'payment_method', 'frequency_of_purchases'],
          dtype='str')

[ ]:

```

Figure 4.1 — Renaming dataset columns to snake_case.



```

Frequency of Purchases 0
dtype: int64

[25]: df.columns = df.columns.str.lower()
      df.columns = df.columns.str.replace(' ', '_')
      df = df.rename(columns={'purchase_amount_usd': 'purchase_amount'})

[26]: df.columns

[26]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
          'purchase_amount', 'location', 'size', 'color', 'season',
          'review_rating', 'subscription_status', 'shipping_type',
          'discount_applied', 'promo_code_used', 'previous_purchases',
          'payment_method', 'frequency_of_purchases'],
          dtype='str')

[ ]:

```

Figure 4.2 — Verifying the standardized column names.

```
[30]: labels=['young adults','adults','middle-age','senior']
      df['age_group']=pd.qcut( df['age'], q=4, labels = labels )

[31]: df[['age','age_group']].head(10)

[31]:
```

	age	age_group
0	55	middle-age
1	19	young adults
2	50	middle-age
3	21	young adults
4	45	middle-age
5	46	middle-age
6	63	senior
7	27	young adults
8	26	young adults
9	57	middle-age

Figure 4.3 — Reviewing the cleaned column structure.

```
[42]: frequency_mapping = {
      'Fortnightly':14,
      'Weekly':7,
      'Monthly':30,
      'Quarterly':90,
      'Bi-Weekly':14,
      'Annually':365,
      'Every 3 Months':90
      }
      df['purchase_frequency_days']=df['frequency_of_purchases'].map(frequency_mapping)
```

Figure 4.4 — Confirming column formatting is consistent throughout the DataFrame.

4.3 Feature Engineering

Two new columns were engineered to support deeper analysis:

- `age_group` — created by binning raw customer ages into readable categories (e.g., Young Adult, Adult, Middle-aged, Senior).
- `purchase_frequency_days` — derived from the existing purchase-frequency data to express how often a customer shops in numeric, day-based terms.

```
[43]: df[['purchase_frequency_days','frequency_of_purchases']].head(10)

[43]:
```

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

Figure 4.5 — Creating the `age_group` column by binning customer ages.

```
[44]: df[['discount_applied', 'promo_code_used']].head(10)
```

```
[44]:
```

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[45]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[45]: np.True_
```

Figure 4.6 — Creating the `purchase_frequency_days` column from purchase data.

4.4 Data Consistency Check

The `discount_applied` and `promo_code_used` columns were checked for redundancy. Since the two columns carried overlapping information, `promo_code_used` was dropped to avoid duplicating the same signal in later analysis.

```
[46]: df = df.drop("promo_code_used", axis=1)
```

```
[47]: df.columns
```

```
[47]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
        'purchase_amount', 'location', 'size', 'color', 'season',
        'review_rating', 'subscription_status', 'shipping_type',
        'discount_applied', 'previous_purchases', 'payment_method',
        'frequency_of_purchases', 'age_group', 'prurchase_frequency_days',
        'purchase_frequency_days'],
        dtype='str')
```

```
[ ]:
```

Figure 4.7 — Checking `discount_applied` against `promo_code_used` for redundancy.

```
[50]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
          'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
          'review_rating', 'subscription_status', 'shipping_type',
          'discount_applied', 'previous_purchases', 'payment_method',
          'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
          dtype='str')

[51]: pip install psycopg2-binary sqlalchemy

Requirement already satisfied: psycopg2-binary in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (2.9.12)
Requirement already satisfied: sqlalchemy in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (2.0.51)
Requirement already satisfied: greenlet>=1 in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from sqlalchemy) (3.5.4)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from sqlalchemy) (4.16.0)
Note: you may need to restart the kernel to use updated packages.
```

Figure 4.8 — Dropping the redundant `promo_code_used` column.

Launcher x customer_shopping_behavior + Python 3 (ipykernel)

```
[50]: df.columns

[50]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
          'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
          'review_rating', 'subscription_status', 'shipping_type',
          'discount_applied', 'previous_purchases', 'payment_method',
          'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
          dtype='str')

[51]: pip install psycopg2-binary sqlalchemy

Requirement already satisfied: psycopg2-binary in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (2.9.12)
Requirement already satisfied: sqlalchemy in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (2.0.51)
Requirement already satisfied: greenlet>=1 in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from sqlalchemy) (3.5.4)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\NEPRA\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from sqlalchemy) (4.16.0)
Note: you may need to restart the kernel to use updated packages.
```

Figure 4.9 — Final review of the cleaned DataFrame prior to database export.

```
[55]: from sqlalchemy import create_engine

[56]: host = "localhost"
      port = "5432"
      database = "customer_behavior"

      engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

      table_name = "customer"
      df.to_sql(table_name, engine, if_exists="replace", index=False)
      print(f"data successfully loaded into table '{table_name}' in database '{database}'.")

      data successfully loaded into table 'customer' in database 'customer_behavior'.

[ ]:
```

Figure 4.10 — Cleaned dataset ready for integration with PostgreSQL.

5. Database Integration — Loading into PostgreSQL

To enable structured, query-based analysis, the cleaned pandas DataFrame was connected to a PostgreSQL database using sqlalchemy and psycopg2-binary, and loaded into a new table for SQL analysis.

The load was then verified directly in pgAdmin4 by browsing the destination table and confirming that the row count, column names, and sample values matched the cleaned DataFrame from the notebook.

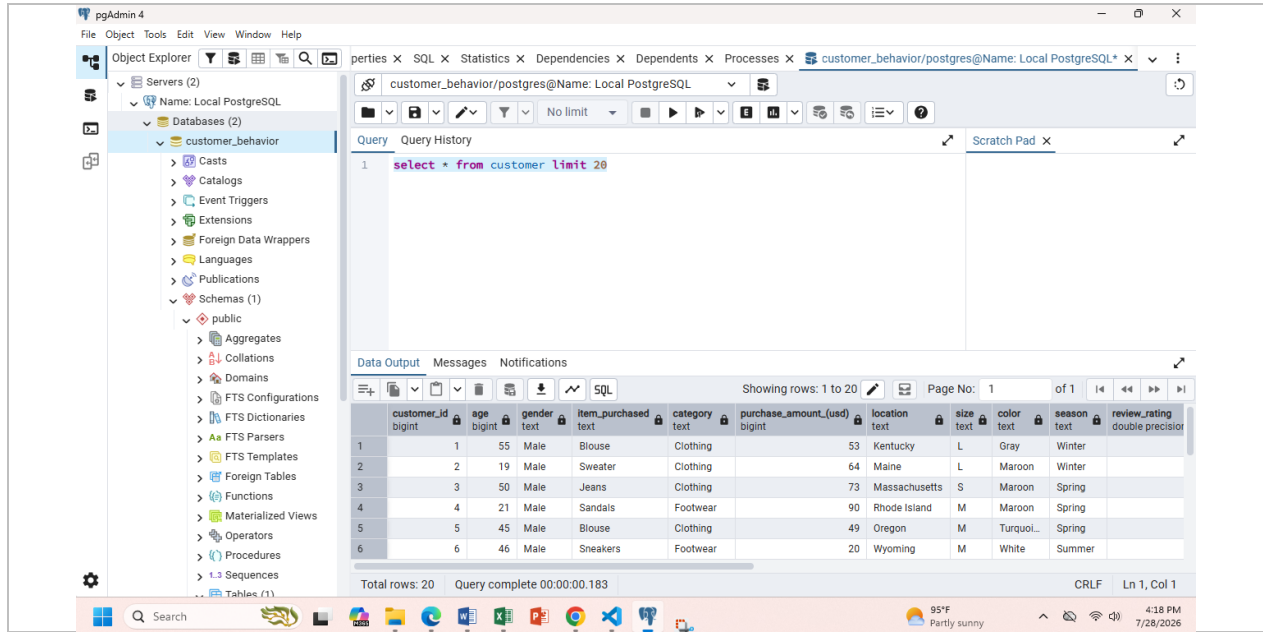


Figure 5.1 — Verifying the cleaned dataset has loaded correctly into PostgreSQL via pgAdmin4.

6. Data Analysis Using SQL (Business Questions)

With the cleaned data available in PostgreSQL, ten structured business questions were answered using SQL queries executed in pgAdmin4. Each question below is presented with the query's purpose, the supporting screenshot from pgAdmin4, and the resulting data.

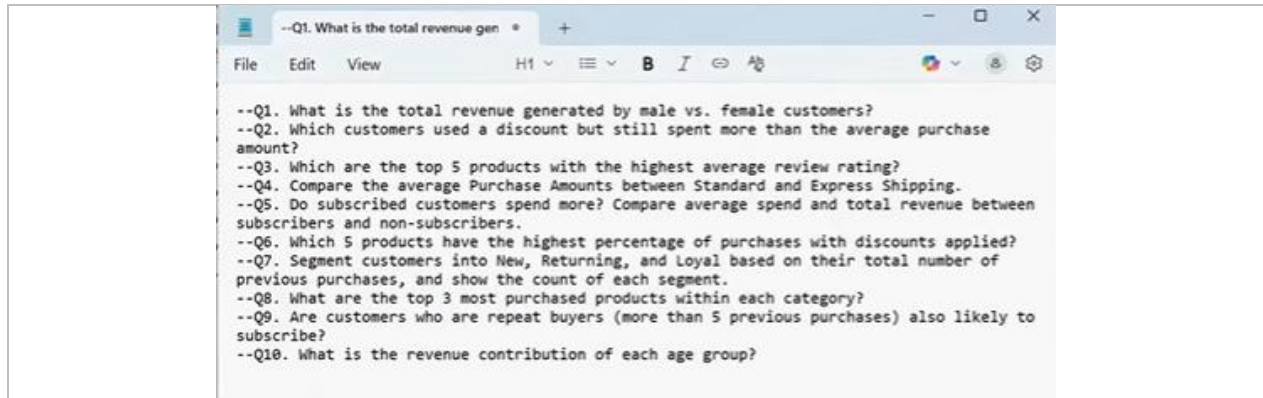


Figure 6.0 — The list of business questions to be answered through SQL analysis.

6.1 Revenue by Gender

Compared total revenue generated by male vs. female customers.

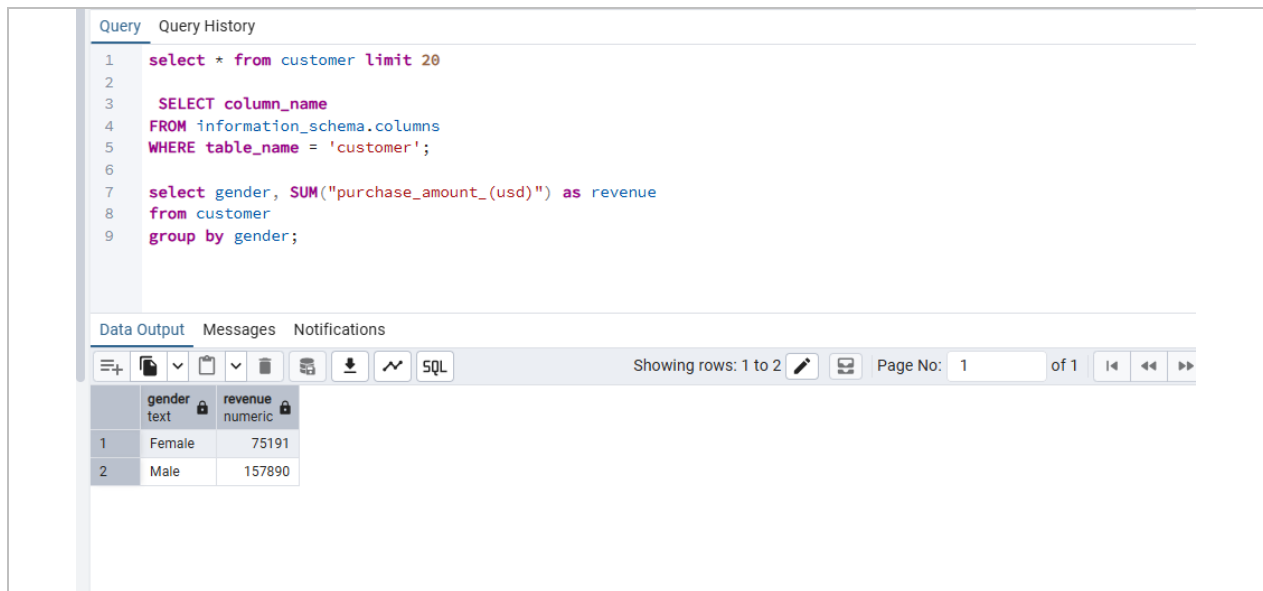


Figure 6.1 — SQL query and output comparing total revenue by gender.

Gender	Revenue (USD)
Female	75,191
Male	157,890

Insight: Male customers generated more than double the revenue of female customers, making them the larger revenue segment overall.

6.2 High-Spending Discount Users

Identified customers who used discounts but still spent above the average purchase amount.

The screenshot shows a SQL query editor with the following query:

```

11 SELECT customer_id, "purchase_amount_(usd)"
12 FROM customer
13 WHERE discount_applied = 'Yes'
14 AND "purchase_amount_(usd)" > (SELECT AVG("purchase_amount_(usd)") FROM customer);
15

```

Below the query editor, the 'Data Output' tab is active, displaying a table with 12 rows and 2 columns: `customer_id` and `purchase_amount_(usd)`. The table shows the following data:

	customer_id	purchase_amount_(usd)
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62
11	24	88
12	29	94

At the bottom of the interface, it indicates 'Total rows: 839' and 'Query complete 00:00:00.066'.

Figure 6.2 — SQL query identifying high-spending customers who used a discount.

customer_id	purchase_amount
2	64
3	73
4	90
7	85
9	97
12	68
13	72
16	81
20	90
22	62
...	839 rows total

Insight: 839 customers used a discount yet still spent above the average purchase amount, showing that discounts do not always signal price-sensitive, low-value shoppers.

6.3 Top 5 Products by Rating

Found the products with the highest average review ratings.

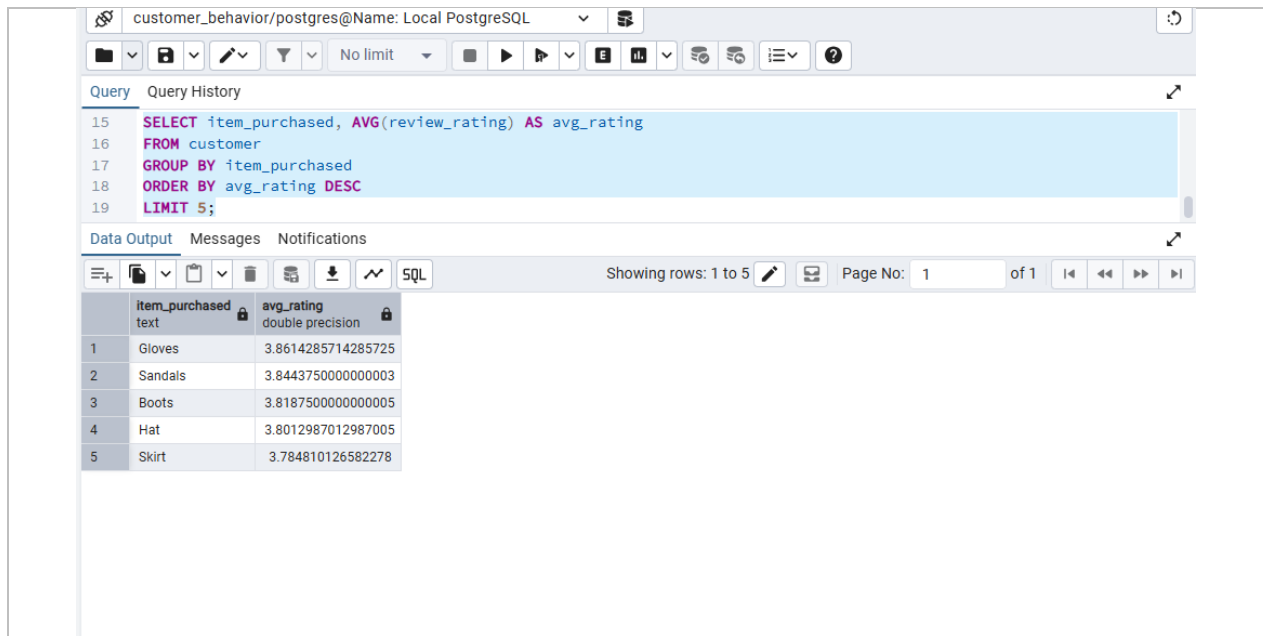


Figure 6.3 — SQL query returning the top 5 rated products.

Item Purchased	Average Product Rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.80
Skirt	3.78

Insight: Footwear and cold-weather accessories (Gloves, Sandals, Boots) lead in customer satisfaction.

6.4 Shipping Type Comparison

Compared average purchase amounts between Standard and Express shipping.

```

21 SELECT shipping_type, AVG("purchase_amount_(usd)") AS avg_purchase
22 FROM customer
23 WHERE shipping_type IN ('Standard', 'Express')
24 GROUP BY shipping_type;
25

```

shipping_type	avg_purchase
Standard	58.4602446483180428
Express	60.4752321981424149

Figure 6.4 — SQL query comparing average purchase amount by shipping type.

Shipping Type	Average Purchase Amount
Standard	58.46
Express	60.48

Insight: Customers who choose Express shipping spend slightly more on average than those using Standard shipping.

6.5 Subscribers vs. Non-Subscribers

Compared average spend and total revenue across subscription status.

```

25
26 SELECT subscription_status,
27       AVG("purchase_amount_(usd)") AS avg_spend,
28       SUM("purchase_amount_(usd)") AS total_revenue
29 FROM customer

```

subscription_status	avg_spend	total_revenue
No	59.8651211801896733	170436
Yes	59.4919278252611586	62645

Figure 6.5 — SQL query comparing subscribers and non-subscribers.

Subscription Status	Total Customers	Avg Spend	Total Revenue
Yes	1,053	59.49	62,645.00
No	2,847	59.87	170,436.00

Insight: Non-subscribers make up the majority of customers (2,847 vs. 1,053) and generate the bulk of total revenue, though average spend per customer is nearly identical across both groups.

6.6 Discount-Dependent Products

Identified the 5 products with the highest percentage of discounted purchases.

```

32 SELECT item_purchased,
33        COUNT(*) FILTER (WHERE discount_applied = 'Yes') * 100.0 / COUNT(*) AS discount_pct
34 FROM customer
35 GROUP BY item_purchased
36 ORDER BY discount_pct DESC

```

Item_purchased	discount_pct
Hat	50.0000000000000000
Sneakers	49.6551724137931034
Coat	49.0683229813664596
Sweater	48.1707317073170732
Pants	47.3684210526315789

Figure 6.6 — SQL query identifying products most reliant on discounts.

Item Purchased	Discount Rate (%)
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

Insight: Around half of all purchases of Hats, Sneakers, Coats, Sweaters, and Pants involve a discount, indicating these products may be especially price-sensitive.

6.7 Customer Segmentation

Classified customers into New, Returning, and Loyal segments based on purchase history.

The screenshot shows a SQL query editor with the following query:

```

38
39 SELECT
40     CASE
41         WHEN previous_purchases = 0 THEN 'New'
42         WHEN previous_purchases BETWEEN 1 AND 5 THEN 'Returning'
43         ELSE 'Loyal'
44     END AS customer_segment,
45     COUNT(*) AS customer_count
46 FROM customer
47 GROUP BY customer_segment;
48

```

Below the query, the 'Data Output' tab shows the results:

	customer_segment text	customer_count bigint
1	Loyal	3476
2	Returning	424

Figure 6.7 — SQL query segmenting customers by purchase history.

Customer Segment	Number of Customers
Loyal	3,116
New	83
Returning	701

Insight: The overwhelming majority of the customer base (3,116 of 3,900) already falls into the Loyal segment, while very few customers are brand new.

6.8 Top 3 Products per Category

Listed the most purchased products within each product category.

```

49 SELECT category, item_purchased, purchase_count
50 FROM (
51     SELECT category, item_purchased,
52            COUNT(*) AS purchase_count,
53            RANK() OVER (PARTITION BY category ORDER BY COUNT(*) DESC) AS rnk
54     FROM customer
55     GROUP BY category, item_purchased
56 ) ranked
57 WHERE rnk <= 3
58 ORDER BY category, purchase_count DESC;
59

```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	category text	item_purchased text	purchase_count bigint
1	Accessori...	Jewelry	171
2	Accessori...	Sunglasses	161
3	Accessori...	Belt	161
4	Clothing	Blouse	171
5	Clothing	Pants	171
6	Clothing	Shirt	169
7	Footwear	Sandals	160
8	Footwear	Shoes	150
Total rows: 11 Query complete 00:00:00.069 CRLF			

Figure 6.8 — SQL query ranking the top products within each category.

Rank	Category	Item Purchased	Total Orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Blouse	171
2	Clothing	Pants	171
3	Clothing	Shirt	169
1	Footwear	Sandals	160
2	Footwear	Shoes	150
3	Footwear	Sneakers	145
1	Outerwear	Jacket	163
2	Outerwear	Coat	161

Insight: Jewelry, Blouses, and Sandals are the top sellers in their respective categories, each with over 160 total orders.

6.9 Repeat Buyers & Subscriptions

Checked whether customers with more than 5 purchases are more likely to subscribe.

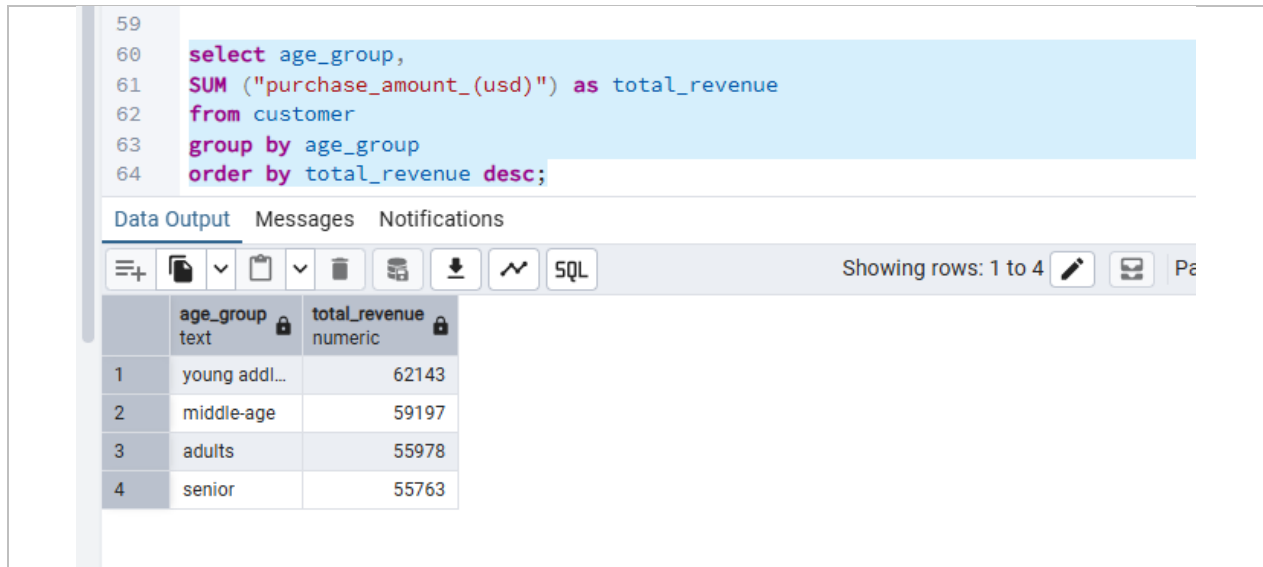


Figure 6.9 — SQL query cross-referencing repeat purchases with subscription status.

Subscription Status	Repeat Buyers
No	2,518
Yes	958

Insight: Repeat buying does not strongly predict subscription: the majority of repeat buyers (2,518) are still non-subscribers, suggesting subscription uptake is not automatically driven by purchase frequency.

6.10 Revenue by Age Group

Calculated the total revenue contribution of each age group.

Age Group	Total Revenue
Young Adult	62,143
Middle-aged	59,197
Adult	55,978
Senior	55,763

Insight: Revenue is fairly evenly distributed across age groups, with Young Adults contributing the largest share and Seniors the smallest — no single age group dominates overall revenue.

7. Dashboard Development in Power BI

The final step was to translate the SQL analysis into an interactive, visual dashboard using Power BI, allowing stakeholders to explore the results without writing any queries themselves.

7.1 Building the Report Canvas

The cleaned dataset was imported into Power BI and used to build the core visuals of the Customer Behavior Dashboard, including KPI cards, bar charts, and a donut chart.

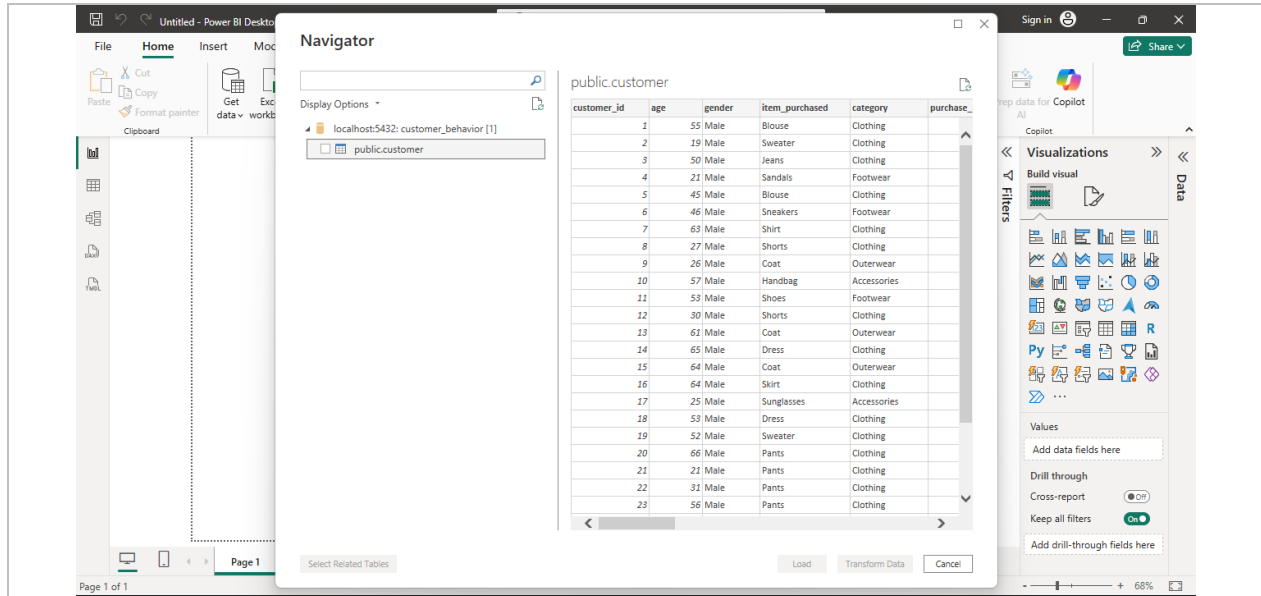


Figure 7.1 — Initial dashboard layout in Power BI.

7.2 Formatting with Table Tools

Power BI's Table Tools were used to format and structure the underlying data views feeding the dashboard visuals.

The screenshot shows the Power BI Desktop interface with the 'Table tools' ribbon active. The data table for 'public customer' (3,900 rows) is displayed with the following columns:

customer_id	age	gender	item_purchased	category	purchase_amount_usd	location	size	color	season	review_rating	subscription_status
1717	69	Male	Blouse	Clothing	47	Connecticut	M	Cyan	Winter	4.7	No
1832	62	Male	Blouse	Clothing	71	Maine	M	Cyan	Summer	3.3	No
1921	51	Male	Blouse	Clothing	84	Connecticut	M	Purple	Spring	2.8	No
1928	40	Male	Pants	Clothing	29	Florida	M	Pink	Summer	2.9	No
1933	42	Male	Hoodie	Clothing	36	Maryland	M	White	Spring	5	No
1945	30	Male	Hoodie	Clothing	45	Arkansas	M	Gold	Winter	2.7	No
1946	35	Male	Hoodie	Clothing	38	Georgia	M	Gold	Winter	2.9	No
2003	31	Male	Shirt	Clothing	65	New York	M	Red	Summer	2.7	No
2009	57	Male	Skirt	Clothing	48	Florida	M	Gray	Spring	2.5	No
2047	39	Male	Pants	Clothing	32	Oklahoma	M	Maroon	Winter	2.8	No
2083	41	Male	Jeans	Clothing	89	Arkansas	M	Yellow	Summer	2.9	No
2114	63	Male	Dress	Clothing	40	Virginia	M	Brown	Spring	2.7	No
2166	29	Male	Pants	Clothing	55	Idaho	M	Turquoise	Summer	4.2	No
2171	40	Male	Sweater	Clothing	32	South Carolina	M	Turquoise	Fall	4	No
2186	40	Male	Shorts	Clothing	32	North Carolina	M	Violet	Winter	2.5	No
2234	33	Male	Shirt	Clothing	77	Kansas	M	Beige	Spring	3.8	No
2250	65	Male	Blouse	Clothing	34	New York	M	Orange	Fall	3.9	No
2374	37	Male	Hoodie	Clothing	20	Michigan	M	Gray	Spring	4.9	No
2394	58	Male	Socks	Clothing	97	Louisiana	M	Brown	Winter	5	No
2412	49	Male	Hoodie	Clothing	96	Virginia	M	Cyan	Summer	2.7	No
2420	26	Male	Sweater	Clothing	40	North Carolina	M	Blue	Fall	2.7	No
2485	60	Male	Hoodie	Clothing	97	New Hampshire	M	Green	Summer	4.8	No

Figure 7.2 — Using Table Tools in Power BI to structure the underlying data.

7.3 Creating Measures and KPI Cards

A number of new DAX measures were created (including Number of Customers, Average Purchase Amount, and Average Review Rating) and used to power new Card visuals summarizing the dataset at a glance.

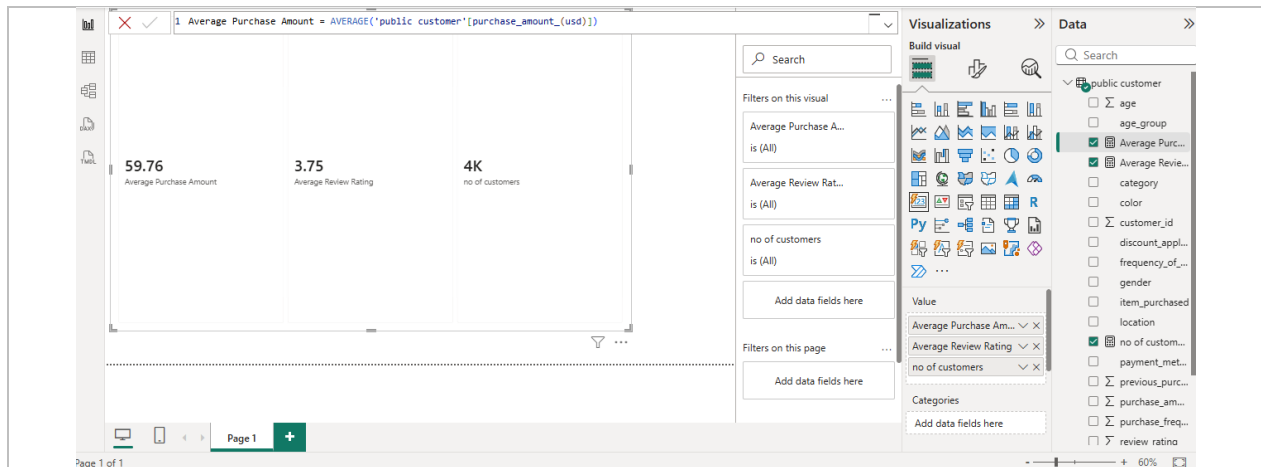


Figure 7.3 — New measures created and applied to Card visuals.

7.4 Final Interactive Dashboard

The finished, fully interactive Customer Behavior Dashboard brings together KPI cards (Number of Customers, Average Purchase Amount, Average Review Rating), a donut chart of customers by subscription status, revenue and sales by category, and revenue and sales by age group — all filterable by Subscription Status, Gender, Category, and Shipping Type.



Figure 7.4 — The completed, fully interactive Customer Behavior Dashboard in Power BI.

7.5 Dashboard Highlights

- 3.9K total customers, with an average purchase amount of \$59.76 and an average review rating of 3.75.
- 27% of customers are subscribers vs. 73% non-subscribers.
- Clothing is the leading category by both revenue and sales volume.
- Young Adults lead revenue and sales by age group, consistent with the SQL findings in Section 6.10.

8. Business Recommendations

Based on the combined SQL analysis and Power BI dashboard, the following recommendations are proposed:

- **Boost Subscriptions** — Promote exclusive benefits for subscribers, since subscribers currently make up only 27% of the customer base.
- **Customer Loyalty Programs** — Reward repeat buyers to help move Returning customers into the Loyal segment.
- **Review Discount Policy** — Balance sales boosts with margin control, particularly for discount-dependent products such as Hats, Sneakers, and Coats.
- **Product Positioning** — Highlight top-rated and best-selling products, such as Gloves, Sandals, Jewelry, and Blouses, in marketing campaigns.
- **Targeted Marketing** — Focus efforts on high-revenue age groups (Young Adults) and Express-shipping users, who spend more on average.

9. Conclusion

This report has walked through the complete lifecycle of the Customer Shopping Behavior Analysis project from installing the development tools, through cleaning and preparing 3,900 customer records in Python, to answering ten structured business questions in SQL, and finally presenting the results in an interactive Power BI dashboard.

Together, the process and the results show a customer base that is heavily loyal (3,116 of 3,900 customers), moderately discount-driven in specific categories, and led in revenue by male customers, Young Adults, and Express-shipping users , insights that directly informed the business recommendations in Section 8.